Interval estimate. μ p or π

Unit 7 Confidence Intervals For Means and Proportions[1]

Learning Objectives

By the end of this lecture, you will be able to:

1. Explain what a confidence interval is
2. Construct CIs for a population mean manually and using t.test() in R
3. Construct CIs for a population proportion manually and using prop.test() in R
4. Calculating the minimum sample size needed for a study
5. Interpret CIs correctly in a computing/tech context

The Core Idea

Suppose you're a data engineer and your team wants to know the average API response time for a new microservice. You can't measure every single request — you collect a sample and use it to estimate the population parameter. μ

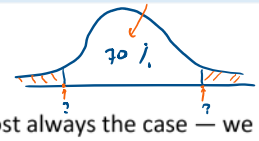
A point estimate (e.g., $\bar{x}$ = 142 ms) gives you a single number but says nothing about how precise it is. A confidence interval gives you a range: a best guess plus an honest statement of uncertainty.

Plain-English Definition

A 95% confidence interval means: if we repeated the sampling process many times and built a CI each time, approximately 95% of those intervals would contain the true population parameter.

It could be loosely interpreted as in our opinion; there is a 95% probability the true value is in this interval.

confidence level



significance level $= \alpha$

$\alpha = 30\%$

CI

Confidence Interval for a Population Mean

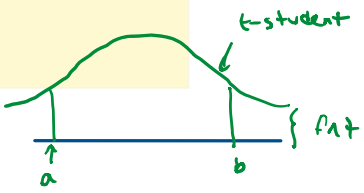
When the population standard deviation (σ) is unknown — which is almost always the case — we use the t-distribution:

μ is unknown

1) σ is unknown
2) n is small

CI for Mean (σ unknown — use this by default)

$$\bar{x} \pm ME = \bar{x} \pm \overbrace{t_{\frac{\alpha}{2},n-1} \times \underbrace{\left(\frac{s}{\sqrt{n}}\right)}_{SE}}^{ME}$$



[1] In making this document, have used Claude.ai Sonnet 4.5 to structure, edit, and create computing related examples.

Symbol	Meaning	How to get it in R
x̄	Sample mean	mean(x)
s	Sample standard deviation	sd(x)
n	Sample size	length(x)
t(α/2, n−1)	Critical t-value for confidence level (1−α)	qt(1 - α/2, df = n−1) or abs(qt(α/2, df = n−1))
$SE = \frac{s}{\sqrt{n}}$	Standard error of the mean	sd(x) / sqrt(length(x))
$ME = t.(SE)$	Margin of error	

When Can You Use This?

- The observations are independent (random sample or randomised experiment)
- Either: n ≥ 30 (Central Limit Theorem kicks in), OR
- The population is approximately normal (check with a Q-Q plot or histogram)

Computing Example — API Response Time

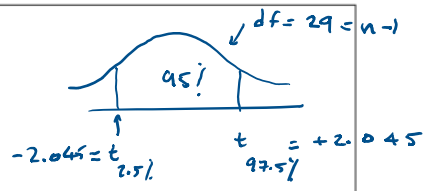
A DevOps team collected response times (ms) for 30 requests to a REST endpoint. They want a 95% CI for the true mean response time. The sample mean is 145ms and the sample standard deviation is 22ms.

```
# --- Simulate some API response time data ---
set.seed(42)
response_times  =  rnorm(30, mean = 145, sd = 22)
# ms

# --- One-line CI ---
result  =  t.test(response_times, conf.level = 0.95)
result$conf.int   # [lower, upper]

# --- Manual calculation (to understand the pieces) ---
n    =  length(response_times)
xbar  =  mean(response_times)
s    =  sd(response_times)
se   =  s / sqrt(n)
t_star  =  qt(0.975, df = n - 1)   # 0.975 = 1 - 0.05/2

lower  =  xbar - t_star * se
upper  =  xbar + t_star * se
cat(sprintf("95%% CI: (%.1f ms, %.1f ms)\n", lower, upper))
```

$n = 30$

$\bar{x} = 145$ ms

$s = 22$ ms



$ME = t_{2.5\%} \cdot SE$

$SE = \frac{s}{\sqrt{n}} = \frac{22}{\sqrt{30}} = 4.0166$

$ME = (2.045)(4.0166) = 8.22$ ms

145 ± 8.22 → UCL = 153.22 ms, → LCL = 136.78 ms

$136.78 < \mu < 153.22$ the 95% CI for μ

Interpretation

Output might be: 95% CI: (136.8 ms, 152.3 ms)

"We are 95% confident that the true mean API response time is between 136.8 ms and 152.3 ms."

Business use: If your SLA requires < 200 ms mean response time, the upper bound of 152.3 ms comfortably satisfies that.

Effect of Sample Size and Confidence Level

Two key controls on interval width:

To get a NARROWER interval...	Trade-off
Increase sample size (n ↑)	More data collection cost / time
Lower confidence level, e.g. 90% instead of 95%	Less confident the interval contains the true value

Using a Known σ (Rare — Mostly Theoretical)

If σ is truly known, use the z-distribution. In practice this almost never applies in computing contexts.

CI for Mean (σ known – theoretical/textbook case)

$$\bar{x} \pm ME = \bar{x} \pm z_{\frac{\alpha}{2}} \times \left(\frac{\sigma}{\sqrt{n}}\right)$$

isolate n in terms of Margin of Error

$$\bar{x} \pm t_{\frac{\alpha}{2}, n-1} \times \frac{s}{\sqrt{n}}$$

We only use this formula to solve for the **minimum required sample size** for a specific margin of error:

$$ME = z_{\frac{\alpha}{2}} \times \frac{\sigma}{\sqrt{n}}, \quad \boxed{n = \left(\frac{z_{\frac{\alpha}{2}} \times \sigma}{ME}\right)^2}$$

Example:

A team wants to estimate mean build time for a CI/CD pipeline to within ±5 seconds with 95% confidence. From a pilot study: s = 30 seconds. What minimum sample size is required?

Given: $ME = 5$ s

$\alpha = 5\%$

$s = 30$ s

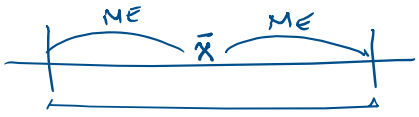
$n = ?$

$$n = \left(\frac{1.96 \times 30}{5}\right)^2 = 139$$

↓

Min sample size.

$$z_{\frac{\alpha}{2}} = \text{abs}(\text{qnorm}(0.025)) = 1.96$$



Confidence Interval for a Population Proportion

$\hat{p}$ = the percentage of success in sample

The Formula — Wald Method

When you have binary outcomes (success/failure, yes/no, error/no-error), you're estimating a proportion P.

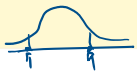
CI for Proportion (Wald / large sample)

qnorm

$$\hat{p} \pm ME = \hat{p} \pm z_{\left(\frac{\alpha}{2}\right)} \times \boxed{\sqrt{\left[\frac{\hat{p}(1-\hat{p})}{n}\right]}}$$

$n\hat{p} \geq 10$ ✓

$n\hat{q} \geq 10$ ✓



Symbol	Meaning
$\hat{p}$	Sample proportion = x / n (successes ÷ total)
n	Sample size
$z_{\frac{\alpha}{2}}$	Critical z-value: 1.645 (90%), 1.96 (95%), 2.576 (99%)
$SE = \sqrt{\left[\frac{\hat{p}(1-\hat{p})}{n}\right]}$	Standard error of the proportion
$ME = z_{\frac{\alpha}{2}}.(SE)$	Margin of Error

When Can You Use This?

The Wald method requires both conditions to hold:

- $n \times \hat{p} \geq 10$ (expected successes)
- $n \times (1-\hat{p}) \geq 10$ (expected failures)

If either fails (e.g. rare events, small samples), use the Wilson score interval: R's prop.test() uses this by default.

Computing Example — Bug Rate in Production

A QA team found 47 bugs in a random sample of 800 code commits. Construct a 95% CI for the true bug rate.

```
result = prop.test(x = 47, n = 800, conf.level = 0.95)
result$conf.int

# Estimated proportion
p_hat = 47 / 800
cat("Bug rate estimate:", round(p_hat * 100, 2),
"%\n")

# --- Manual Wald CI (for understanding) ---
z_star = qnorm(0.975)  # 1.96
se_p = sqrt(p_hat * (1 - p_hat) / 800)
lower = p_hat - z_star * se_p
upper = p_hat + z_star * se_p
cat(sprintf("95%% Wald CI: (%.3f, %.3f) = (%.1f%%,
%.1f%%)\n",
        lower, upper, lower*100, upper*100))
```

$n = 800$

$\hat{p} = \frac{47}{800}$

$\hat{q} = 1 - \frac{47}{800}$

$SE = \sqrt{\left(\frac{\hat{p}\hat{q}}{n}\right)}$

$=$

$\alpha = 5\%$

$|z_{0.025}| = |z_{0.975}| = \text{abs}(\text{qnorm}(0.025))$

$= 1.96$

$4.2\% < P < 7.5\%$ the 95% CI for P

Interpretation

Output might be: 95% CI: (0.043, 0.072) → (4.3%, 7.2%)

"We are 95% confident that the true bug rate for all commits is between 4.3% and 7.2%."

Business use: Management sets a threshold of < 10% bug rate. The entire CI is below 10%, so the codebase likely meets the quality standard.

Wald vs Wilson — Which to Use?

Wald (manual formula)	Wilson Score (prop.test default ✓)
Simple, fast to compute	Better coverage for small n or extreme p
Requires $n \cdot \hat{p} \geq 10$ and $n \cdot (1-\hat{p}) \geq 10$	Works well even when n is small
OK for teaching intuition	Use this in practice — R's default

```
# To get Wald CI explicitly:
p_hat  =  47 / 800
z  =  qnorm(0.975)
se  =  sqrt(p_hat * (1 - p_hat) / 800)
wald_ci  =  c(p_hat - z*se, p_hat + z*se)

# Compare:
wilson_ci  =  prop.test(47, 800)$conf.int
cat("Wald:  ", round(wald_ci, 4), "\n")
cat("Wilson:", round(wilson_ci, 4), "\n")
```

Finding the sample size for a fixed Margin of error:

$$ME = z_{\frac{\alpha}{2}} \sqrt{\frac{\hat{p}\hat{q}}{n}}$$

prior

$$n = \frac{\left(z_{\frac{\alpha}{2}}\right)^2 \hat{p}\hat{q}}{(ME)^2}$$

If no prior is available: $\hat{p} = \hat{q} = 50\%$

Example A — Mean CPU Usage (Large n)

$n = 200$ $\bar{x} = 63$

A cloud platform monitors CPU utilisation (%) across 200 randomly sampled VMs. The sample mean is 63 and the ~~population mean~~ is 18. Build a 99% CI for mean CPU usage.

Sample $s = 18$ → $\alpha = 1\%$

```
set.seed(7)
cpu  =  rnorm(200, mean = 63, sd = 18)   # simulated
CPU usage

ci99  =  t.test(cpu, conf.level = 0.99)$conf.int
cat(sprintf("99%% CI for mean CPU: (%.1f%%,
%.1f%%)\n", ci99[1], ci99[2]))

# Diagnostic: check normality assumption
par(mfrow = c(1, 2))
hist(cpu, main = "CPU Utilisation", col = "steelblue",
xlab = "%")
qqnorm(cpu); qqline(cpu, col = "red")
par(mfrow = c(1, 1))
```

Given abs(qt(0.005, 199)) = 2.6

$$SE = \frac{S}{\sqrt{n}} = \frac{18}{\sqrt{200}} = 1.27$$

$$ME = \left(t_{\frac{\alpha}{2}, n-1}\right)(SE) = (2.6)(1.27)$$

$ME = 3.31$

$\bar{x} \pm ME$ → UCL = 66.3, → LCL = 59.69

$59.7 < \mu < 66.3$ percent

Example B — Proportion of Failed Logins

A security analyst samples 1,200 login attempts and finds 84 failed authentication. Build a 95% CI for the failure rate. Are we confident the true rate is below 10%?

the true fail rate is $5.6\% < P < 8.4\%$

```
x = 84; n = 1200

# Check conditions
p_hat = x / n
cat("p_hat:", round(p_hat, 4),
  " n*p̂ =", n*p_hat,
  " n*(1-p̂) =", n*(1-p_hat), "\n")

# Wilson CI
result = prop.test(x, n, conf.level = 0.95)
ci = result$conf.int
cat(sprintf("95%% CI: (%.3f, %.3f) = (%.1f%%,
%.1f%%)\n",
        ci[1], ci[2], ci[1]*100, ci[2]*100))

# Is the entire CI below 10%?
cat("Upper bound < 10%?", ci[2] < 0.10, "\n")
```

$n = 1200$

$\hat{p} = \frac{84}{1200}$, $\hat{q} = 1 - \hat{p}$ $\alpha = 5\%$

$Z_{\frac{\alpha}{2}} = \text{abs}(\text{qnorm}(0.025)) = 1.96$

$SE = \sqrt{\left(\frac{\frac{84}{1200} \times \frac{1116}{1200}}{1200}\right)} = 0.0074$

$ME = (1.96)(0.0074) = 0.014$

$\hat{p} \pm ME$ → UCL = 8.4 %
→ LCL = 5.6 %

Yes, we are confident that P is less than 10% since UCL and LCL are both less than 10%

Example C — Comparing CIs Across Confidence Levels

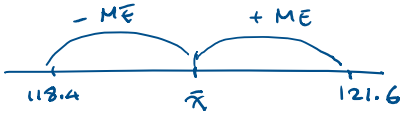
```
set.seed(42)
latency = rnorm(50, mean = 210, sd = 35)  # network
latency ms

levels = c(0.90, 0.95, 0.99)
for (cl in levels) {
  ci = t.test(latency, conf.level = cl)$conf.int
  cat(sprintf("%d%% CI: (%.1f, %.1f)  width = %.1f
ms\n",
          cl*100, ci[1], ci[2], diff(ci)))
}

# Example output:
# 90% CI: (202.1, 219.4)  width = 17.3 ms
# 95% CI: (200.2, 221.3)  width = 21.1 ms
# 99% CI: (196.3, 225.2)  width = 28.9 ms
```

Example D - Working Backwards:

$\alpha = 5\%$

A study measured the **average latency of API requests**. From a sample of **64 requests**, a **95% confidence interval** for the mean latency was found to be:

(118.4,121.6) milliseconds



Answer the following:

1. What is the **sample mean latency**? 120 ms
2. What is the **margin of error**? 1.6 ms
3. What is the **standard error of the mean**? 0.8 ms
4. What is the **sample standard deviation**? S = ms
5. Interpret the confidence interval in context.

$$\bar{x} = \frac{121.6 + 118.4}{2} = 120 \text{ ms}$$

$$ME = \frac{121.6 - 118.4}{2} = 1.6 \text{ ms}$$

$$SE = \frac{S}{\sqrt{n}} = ?$$

$$t_{\frac{\alpha}{2}, n-1} = \text{abs}(\text{qt}(0.025, 63))$$

$$= 1.998$$

$$0.8 = \frac{S}{\sqrt{64}}, \quad S = 6.4 \text{ ms}$$

$$ME = \left(t_{\frac{\alpha}{2}, n-1}\right) \cdot (SE)$$

$$\frac{1.6}{1.998} = SE, \quad SE = 0.8$$

We are 95% confident that the true population average latency of API requests is between 118.4 and 121.6 ms

Key Takeaways

- ✓ A CI quantifies estimation uncertainty — always report one alongside a point estimate.
- ✓ Use t.test() for means (handles unknown σ automatically with t-distribution).
- ✓ If R available, use prop.test() for proportions (Wilson score, works for small n too).
- ✓ More data → narrower CI. Higher confidence level → wider CI.
- ✓ Interpret carefully: the confidence is in the method, not in any single interval.
- ✓ Always check your assumptions: independence, normality (for small n), and $n \cdot \hat{p} \geq 10$.